Underworld3

Louis Moresi [1]

[1] Australian National University

DOI https://doi.org/10.6084/m9.figshare.33193533

Abstract

Introducing Underworld3: Mathematically Self-Describing Modelling in Python for Desktop, HPC and Cloud.

Keywords Underworld Code, petsc

The most recent version of Underworld (v3), is a ground-up rewrite of our widely-used geodynamics modelling code. I don't think there is a single line of code carried forward from v2. Underworld3 does share many ideas with Underworld2: it is a finite element code in which particle swarms and mesh variables have equal standing, with a python front end and a parallel-optimised (PETSc) back end, with rich symbolic algebra capabilities to help set up the mathematics of your model (this time we are using sympy), and full integration with the Jupyter notebook system for designing and analysing problems.

We say that Underworld3 is mathematically self-describing because every data object has a symbolic (sympy) equivalent, all the equation systems retain their symbolic representation throughout the problem assembly and, as a result, when you have finished building your problem, you can ask to see the mathematical representation of the numerical implementation. The symbolic representation that Underworld uses makes it easy to evaluate model derivatives for non-linear problems and optimisations, and it also means that you can view, check, simplify and re-use those derivatives in a mathematical form later in your scripts.

There is very little performance penalty for providing this mathematical introspection because sympy compiles PETSc-compatible C-functions when they are needed. This extra compilation step is offset by the fact that sympy can simplify expressions, combine terms, and eliminate common identities before compilation (this is particularly useful when computing Jacobians).

1. How does this look in practice ?

Underworld3 provides python templates for a number of equations such as Stokes or Advection-Diffusion, which do two things:

PUBLISHED Dec 08, 2024 ARTICLE UWTN 2024-002 VERSION 1.0.0 LICENCE CC-BY-4.0 ARCHIVED 2026-08-10T04:19:43Z SOURCE https://www.underworldcode.org/underworld3-come-and-get-it/

- Set up the general equation-solving infrastructure for the equation you want to solve, validate the variables (define new variables if they are not provided), add auxiliary solvers where coupled sub-equations are part of the problem, and so on. For example, in advection-diffusion problems, the advection part of the solver has a number of sub-components that require orchestration behind-the-scenes.
- Provide a number of hooks that the template requires you to fill out in order to solve the problem. For example, Stokes requires an expression for the body force, the addition of a constitutive model and its material properties, boundary conditions which may be functions of the unknowns.

The solver classes are self-describing. We can `view()` the class description before created a solver to see if this is what we need:

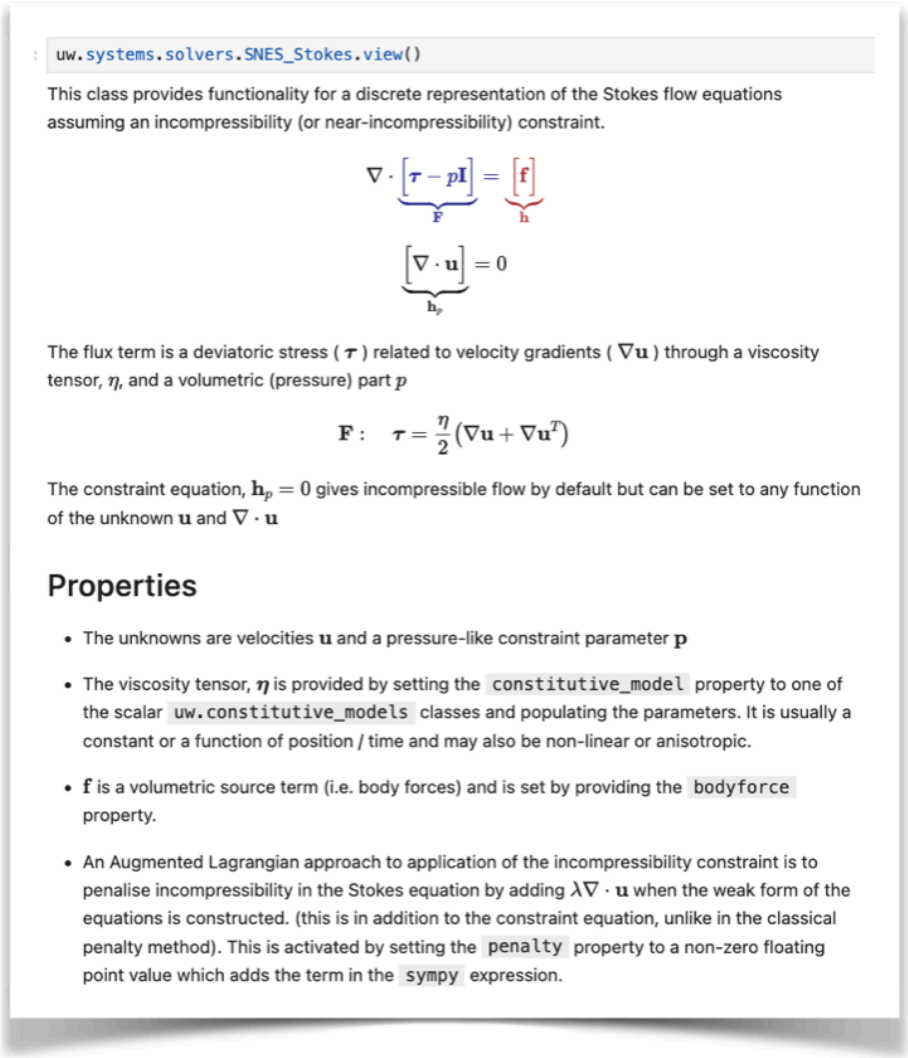
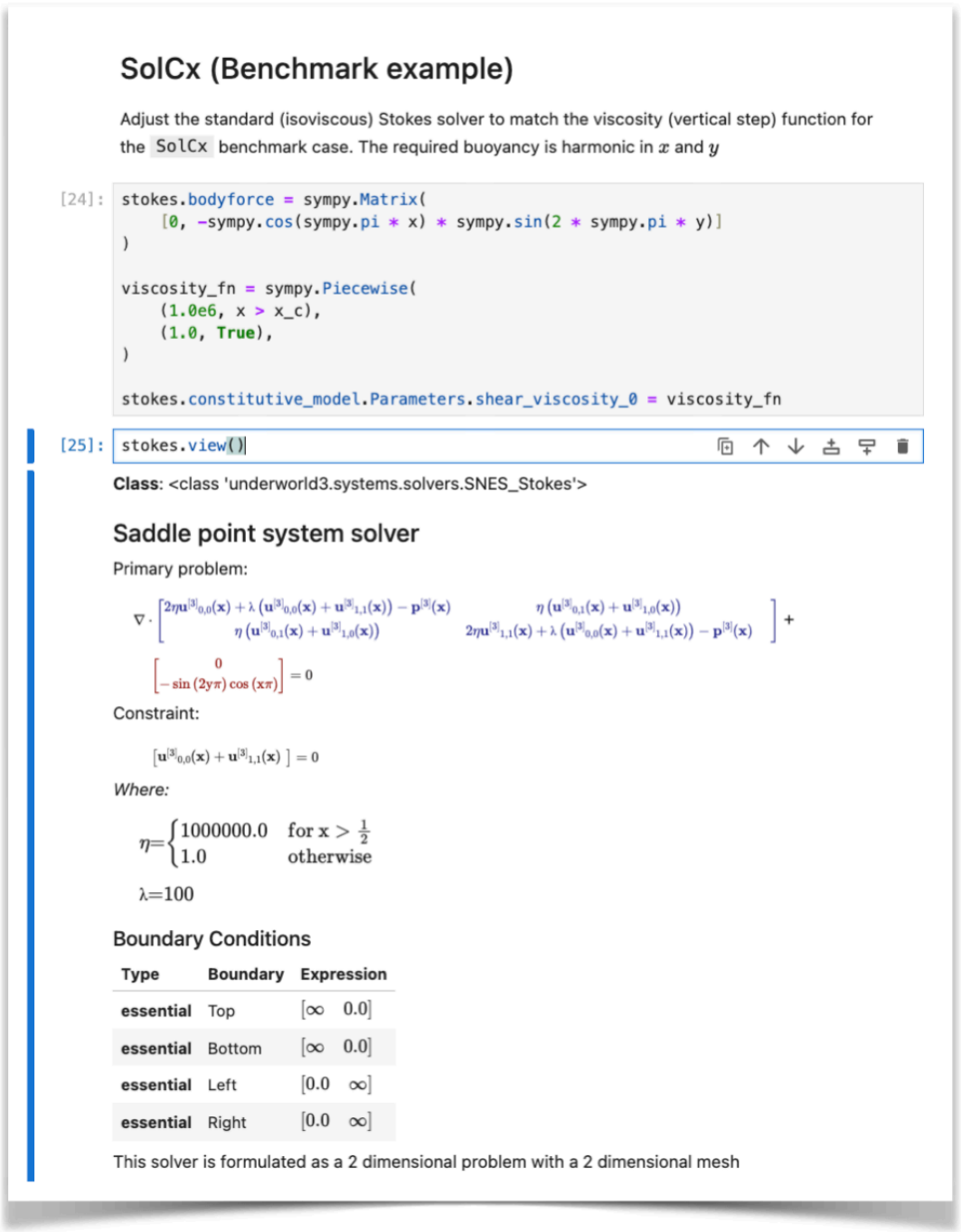


Figure 1: A Jupyter notebook in which we ask the Stokes solver class to describe itself before we decide if this is what we want to use, and it also tells us what variables we need (or will be given) when we set it up.

In a notebook, the Stokes problem is set up with some simple python code, most of which is to provide suitable expressions for the constitutive parameters, boundary conditions and the right-hand-side. When we view the solver **object** (rather than the class), it shows the mathematical description of how it has been set up. This is the superficial description, if you want to find out what the Jacobians look like, you can dig a bit deeper and find out !



SolCx (Benchmark example)

Adjust the standard (isoviscous) Stokes solver to match the viscosity (vertical step) function for the `SolCx` benchmark case. The required buoyancy is harmonic in x and y

[24]:
```
stokes.bodyforce = sympy.Matrix(
    [0, -sympy.cos(sympy.pi * x) * sympy.sin(2 * sympy.pi * y)]
)

viscosity_fn = sympy.Piecewise(
    (1.0e6, x > x_c),
    (1.0, True),
)

stokes.constitutive_model.Parameters.shear_viscosity_0 = viscosity_fn
```

[25]:
```
stokes.view()
```

Class: <class 'underworld3.systems.solvers.SNES_Stokes'>

Saddle point system solver

Primary problem:

$$\nabla \cdot \begin{bmatrix} 2\eta \mathbf{u}^{[3]}_{0,0}(\mathbf{x}) + \lambda\left(\mathbf{u}^{[3]}_{0,0}(\mathbf{x}) + \mathbf{u}^{[3]}_{1,1}(\mathbf{x})\right) - \mathbf{p}^{[3]}(\mathbf{x}) & \eta\left(\mathbf{u}^{[3]}_{0,1}(\mathbf{x}) + \mathbf{u}^{[3]}_{1,0}(\mathbf{x})\right) \\ \eta\left(\mathbf{u}^{[3]}_{0,1}(\mathbf{x}) + \mathbf{u}^{[3]}_{1,0}(\mathbf{x})\right) & 2\eta \mathbf{u}^{[3]}_{1,1}(\mathbf{x}) + \lambda\left(\mathbf{u}^{[3]}_{0,0}(\mathbf{x}) + \mathbf{u}^{[3]}_{1,1}(\mathbf{x})\right) - \mathbf{p}^{[3]}(\mathbf{x}) \end{bmatrix} +$$

$$\begin{bmatrix} 0 \\ -\sin(2y\pi)\cos(x\pi) \end{bmatrix} = 0$$

Constraint:

$$\left[\mathbf{u}^{[3]}_{0,0}(\mathbf{x}) + \mathbf{u}^{[3]}_{1,1}(\mathbf{x})\right] = 0$$

Where:

$$\eta = \begin{cases} 1000000.0 & \text{for } \mathbf{x} > \frac{1}{2} \\ 1.0 & \text{otherwise} \end{cases}$$

$$\lambda = 100$$

Boundary Conditions

Type	Boundary	Expression
essential	Top	$[\infty \quad 0.0]$
essential	Bottom	$[\infty \quad 0.0]$
essential	Left	$[0.0 \quad \infty]$
essential	Right	$[0.0 \quad \infty]$

This solver is formulated as a 2 dimensional problem with a 2 dimensional mesh

Figure 2: A Jupyter notebook in which we set up a Stokes solver for a typical benchmark problem which has a spatially variable buoyancy force term, and a vertical step in viscosity.

2. Why do I need the mathematical description, really ?

With Underworld3 we haven't attempted to hide away all the mathematical complexity that you need to know to do a good job of numerical modelling, so why do we think it is helpful to add this level of "self-description" ?

The use of `sympy` is to let the equations simplify and combine terms (for example, it automatically degenerates a viscoelastic problem to a viscous one if the elastic moduli vanish) and we think that you should be aware of what it is doing.

Here is an example: in this optimisation problem, we need to compute derivatives of the objective function (sorry, "funtion" !) and we also clone the original solver with new variables. The validation might just be a visual inspection of the solver, but you can also grab the terms individually and simplify or substitute into them to check they are correct.

Compute the objective funtion and its derivatives (adjoint RHS f

The adjoint of the forward problem is used to compute the changes to the forward problem that will redu
the path we take to reduce the misfit is always constrained to follow valid solutions to the forward proble

The right hand side of the adjoint equation comes from the derivative of J the cost function with respect
from J. In this case, J only depends on the velocity unknown.

```
# This is the force term for the adjoint velocity equation
dJ_dv = uw.function.derivative(J, v_soln1.sym)

# This is the force term for the adjoint pressure / constraint equation
dJ_dp = uw.function.derivative(J, v_soln1.sym)
```

```
dJ_dv
```

```
dJ_dp
```

$$\left[\frac{2V1_0(\mathbf{x})-2Vo_0(\mathbf{x})}{100(Vo_0^2(\mathbf{x})+Vo_1^2(\mathbf{x})+\frac{1}{100000})}+V1_0(\mathbf{x})-Vo_0(\mathbf{x}) \quad \frac{2V1_1(\mathbf{x})-2Vo_1(\mathbf{x})}{100(Vo_0^2(\mathbf{x})+Vo_1^2(\mathbf{x})+\frac{1}{100000})}+V1_1(\mathbf{x})-Vo_1(\mathbf{x})\right]$$

Check the adjoint solver is properly constructed

We should check that the form of the solver is correct, that the adjoint variables have been appropriatel
objective function is added to this term correctly.

The entire equation system can be seen using

```
stokes_adjoint.view()
```

which reports the equation structure (with simplification and combination of terms). Sub-expressions su
conditions are listed in a table.

```
stokes_adjoint.view()
```

Class: <class 'underworld3.systems.solvers.SNES_Stokes'>

Saddle point system solver

Primary problem:

$$\nabla\cdot\begin{bmatrix}2\eta U1_{0,0}(\mathbf{x})+\lambda\left(U1_{0,0}(\mathbf{x})+U1_{1,1}(\mathbf{x})\right)-Q1(\mathbf{x}) & \eta\left(U1_{0,1}(\mathbf{x})+U1_{1,0}(\mathbf{x})\right)\\ \eta\left(U1_{0,1}(\mathbf{x})+U1_{1,0}(\mathbf{x})\right) & 2\eta U1_{1,1}(\mathbf{x})+\lambda\left(U1_{0,0}(\mathbf{x})+U1_{1,1}(\mathbf{x})\right)-Q1(\mathbf{x})\end{bmatrix}+$$

$$\left[\beta_{\max}\left(\frac{\tanh\left(10\beta(\mathbf{x})\right)}{2}+\frac{1}{2}\right)U1_0(\mathbf{x})-\frac{2V1_0(\mathbf{x})-2Vo_0(\mathbf{x})}{100(Vo_0^2(\mathbf{x})+Vo_1^2(\mathbf{x})+\frac{1}{100000})}-V1_0(\mathbf{x})+Vo_0(\mathbf{x}) \quad \beta_{\max}\left(\frac{\tanh\left(10\beta(\mathbf{x})\right)}{2}+\frac{1}{2}\right)U1_1(\mathbf{x})-\frac{2V1_1(\mathbf{x})-2Vo_1(\mathbf{x})}{100(Vo_0^2(\mathbf{x})+Vo_1^2(\mathbf{x})+\frac{1}{100000})}\right]$$

Constraint:

$$\left[U1_{0,0}(\mathbf{x})+U1_{1,1}(\mathbf{x})\right]=0$$

Where:

$$\lambda=1$$

$$\beta_{\max}=100000$$

$$\eta=\frac{99\tanh\left(10\beta(\mathbf{x})\right)}{2}+\frac{101}{2}$$

3. How do I get started ?

You can also check the Underworld3 Quickstart guide to browse example notebooks and see how to install the code. Before you install anything, though, you might like to try running the example notebooks on mybinder.org.

4. Go Git the Code !

It's all available on through GitHub at https://github.com/underworldcode/underworld3

Feature image: *Underworld3 uses unstructured meshes supplied by gmsh or any other tool supported by PETSc. In this image we see flow driven from the left boundary around a series of obstacles visualised by streamlines. The intense colours appear where the flow is strongest which is also where the gaps between obstacles line up. There is an emergent pressure gradient displayed in the background shading.*

The Underworld project is supported by AuScope and the Australian Government through the National Collaborative Research Infrastructure Strategy (NCRIS). Source code: github.com/underworldcode/underworld3